

交易解释器 MVP 需求文档

1. 项目目标

开发一个“交易解释器”的最小可用版本。用户输入一笔区块链交易哈希后，系统自动读取该交易的链上详情、事件日志和相关合约 ABI，并调用 LLM 生成一段面向普通用户的交易解释。

本版本只做“读取与解释”，不做钱包连接、不做签名、不发起交易、不提供投资建议。

2. MVP 范围

2.1 支持范围

第一版优先支持：

- Ethereum 主网或 Sepolia 测试网，二选一即可
- 输入交易哈希
- 读取交易详情
- 读取交易 receipt
- 读取事件日志 logs
- 获取相关合约 ABI
- 解码交易 input data
- 解码事件日志
- 调用 LLM 生成自然语言解释
- 输出 Markdown 格式结果

2.2 暂不支持

第一版不做：

- 钱包连接
 - 用户签名
 - 自动发交易
 - 多链自动识别
 - 复杂 DeFi 交易完整路径还原
 - 风险评分
 - 合约安全审计
 - 资产价格估算
 - 税务、投资、法律建议
-

3. 用户流程

1. 用户输入一笔交易哈希。
2. 系统根据配置的链读取交易详情。
3. 系统读取交易 receipt 和事件日志。
4. 系统收集交易中涉及的地址，包括：
5. 交易发起地址 from
6. 交易接收地址 to
7. 事件日志中的合约地址
8. ERC20 / ERC721 / ERC1155 Transfer 事件中涉及的地址
9. 系统尝试获取相关合约 ABI。
10. 系统使用 ABI 解码：
11. 交易调用的方法
12. 方法参数
13. 事件日志
14. 系统整理成结构化数据。
15. 系统将结构化数据发送给 LLM。
16. LLM 生成交易解释。
17. 系统返回解释结果。

4. 输入要求

4.1 最小输入

用户输入：

0x 开头的交易哈希

示例：

0xc2f3098906d406dfef5511711ef0c86ecfe233a54242af79060bf8807ee5b73c

4.2 配置项

系统通过环境变量配置：

```
CHAIN=sepolia
RPC_URL=
BLOCK_EXPLORER_API_KEY=
LLM_BASE_URL=
LLM_API_KEY=
LLM_MODEL=
```

5. 数据读取要求

5.1 交易详情

系统需要读取以下链上字段：

- transaction hash
- block number
- block timestamp
- from address
- to address
- value
- input data
- gas
- gas price
- max fee per gas, 如存在
- max priority fee per gas, 如存在
- nonce
- chain id

5.2 Receipt 信息

系统需要读取：

- transaction status
- gas used
- effective gas price
- contract address, 如为合约部署交易
- logs
- cumulative gas used

5.3 事件日志

系统需要读取每条 log 的：

- log address
- topics

- data
- log index
- transaction index
- removed 状态，如存在

6. ABI 获取规则

系统需要按以下优先级获取 ABI：

1. 优先从区块浏览器 API 获取已验证合约 ABI。
2. 如果 ABI 获取失败，保留原始 input data 和 logs。
3. 对常见标准事件内置 ABI，至少包括：
4. ERC20 Transfer
5. ERC20 Approval
6. ERC721 Transfer
7. ERC721 Approval
8. ERC1155 TransferSingle
9. ERC1155 TransferBatch

当 ABI 无法获取时，系统不得编造方法名或事件含义，只能说明“该合约 ABI 未获取到，无法完全解码”。

7. 解码要求

7.1 交易 input 解码

如果 to 地址对应的合约 ABI 可用，需要解析：

- 调用的合约方法名
- 方法参数名称
- 方法参数值

示例结构：

```
{
  "methodName": "transfer",
  "params": {
    "to": "0x...",
    "amount": "1000000000000000000"
  }
}
```

```
}  
}
```

7.2 事件日志解码

如果事件 ABI 可用，需要解析：

- event name
- event params
- emitting contract address

示例结构：

```
{  
  "eventName": "Transfer",  
  "contractAddress": "0x...",  
  "params": {  
    "from": "0x...",  
    "to": "0x...",  
    "value": "1000000000000000000"  
  }  
}
```

8. LLM 输出要求

LLM 最终输出必须包含以下部分：

交易解释

1. 用户发起了什么动作

用普通人能理解的语言说明该交易大概做了什么。

例如：

- 用户向某地址转账
- 用户调用了某个合约方法
- 用户授权某个合约使用自己的代币
- 用户与 DeFi 合约发生交互
- 用户领取、铸造、交换或销毁某种资产

如果无法确定，需要明确写出“不确定”。

2. 涉及哪些资产和地址

列出：

- 发起地址
- 接收地址
- 被调用合约地址
- 涉及的代币合约地址
- 资产类型
- 数量
- 收款方
- 付款方

如果代币名称、符号或精度无法获取，需要保留原始链上数值，并说明未能确认。

3. 哪些信息来自链上数据

明确列出来自链上数据的事实，例如：

- 交易哈希
- from 地址
- to 地址
- value
- status
- gas used
- block number
- decoded method
- decoded events
- Transfer / Approval 事件

这一部分必须只写链上可验证信息，不允许模型自由推断。

4. 哪些信息是模型推断

明确列出模型根据链上数据推断出的内容，例如：

- 这可能是一次 ERC20 转账
- 这可能是一次授权操作
- 这可能是一次 swap
- 这可能是一次 NFT 转移
- 这可能是某个协议交互

推断必须使用“可能”“看起来像”“根据事件日志推测”等措辞，不得说成确定事实。

5. 模型不确定的地方

列出所有不确定信息，例如：

- 合约 ABI 不完整
- 事件无法解码
- 无法确认合约真实业务含义
- 无法确认地址归属
- 无法确认用户真实意图
- 无法判断交易是否安全
- 无法判断资产价格或实际收益

6. 如果要签类似交易，用户应该检查什么

给出签名前检查清单，包括：

- 确认目标地址是否正确
- 确认调用的是哪个合约
- 确认授权额度是否过大
- 确认转出的资产类型和数量
- 确认交易是否包含 approve、permit、setApprovalForAll 等授权行为
- 确认 gas 费用
- 确认交易所在网络
- 确认是否在可信前端或可信钱包中发起
- 如果是复杂 DeFi 交易，确认滑点、最低收到数量、截止时间和交易路径

9. 输出格式示例

```
# 交易解释
```

```
## 1. 用户发起了什么动作
```

这笔交易由地址 ``0x...`` 发起，调用了合约 ``0x...`` 的 ``transfer`` 方法。根据解码结果，用户可能是在向另一个地址转移某种 ERC20 代币。

```
## 2. 涉及哪些资产和地址
```

- 发起地址：``0x...``
- 接收地址：``0x...``
- 合约地址：``0x...``
- 资产类型：疑似 ERC20 Token
- 转移数量：链上原始数值为 ``1000000000000000000``
- 代币符号：未确认

3. 来自链上数据的信息

- 交易哈希：`0x...`
- 交易状态：成功
- 区块高度：`123456`
- 调用方法：`transfer`
- 事件日志中包含 `Transfer` 事件
- gas used：`xxxxx`

4. 模型推断的信息

- 根据方法名和事件日志，这笔交易看起来像一次 ERC20 代币转账。
- 由于未获取到完整代币元数据，暂时不能确认该资产的名称和符号。

5. 不确定的地方

- 无法确认该代币的真实项目背景。
- 无法确认接收地址的实际归属。
- 无法判断这笔交易是否来自用户主动操作，还是由某个前端引导完成。

6. 签类似交易前应该检查什么

- 检查接收地址是否正确。
- 检查转出的资产和数量是否符合预期。
- 检查是否存在额外授权行为。
- 检查 gas 费用是否合理。
- 检查当前网络是否正确。

10. LLM 提示词要求

系统传给 LLM 的 prompt 必须包含：

你是一个区块链交易解释助手。你的任务是基于提供的链上结构化数据解释交易含义。

要求：

1. 只能把链上字段中明确存在的信息称为事实。
2. 对无法直接从链上确认的信息，必须标记为“模型推断”或“不确定”。
3. 不要编造合约名称、项目名称、地址归属或用户意图。
4. 不要提供投资建议。
5. 不要承诺交易安全。
6. 输出必须包含：
 - 用户发起了什么动作
 - 涉及哪些资产和地址
 - 哪些信息来自链上数据

- 哪些信息是模型推断
- 模型不确定的地方
- 如果要签类似交易，用户应该检查什么

11. 技术实现建议

建议使用：

- Node.js
- TypeScript
- viem 或 ethers.js
- 区块浏览器 API 获取 ABI
- OpenAI-compatible LLM API，由 Hermes 负责调用本地或远程模型

推荐目录结构：

```
tx-explainer/  
  src/  
    index.ts  
    chains.ts  
    fetchTransaction.ts  
    fetchAbi.ts  
    decodeInput.ts  
    decodeLogs.ts  
    buildLLMPrompt.ts  
    callLLM.ts  
    formatOutput.ts  
  .env.example  
  package.json  
  README.md
```

12. CLI 使用方式

第一版可以先做 CLI，不做网页。

示例命令：

```
npm run explain -- --tx 0x交易哈希
```

输出：

13. 验收标准

MVP 完成后，需要满足：

1. 输入一个有效交易哈希后，可以成功读取交易详情。
2. 可以读取交易 receipt 和 logs。
3. 可以尝试获取相关合约 ABI。
4. 对已验证合约，可以解码方法名和事件。
5. 对无法解码的交易，不得编造解释。
6. LLM 输出必须区分：
 7. 链上事实
 8. 模型推断
 9. 不确定信息
10. 输出中必须包含签名前检查清单。
11. 整个流程不需要连接钱包，也不会发起任何交易。

14. 给 Hermes 的实现任务

请基于以上需求，实现一个“交易解释器”最小版本。

优先目标：

1. 使用 TypeScript + Node.js 实现 CLI。
2. 支持通过交易哈希读取链上交易详情。
3. 支持读取 receipt 和 logs。
4. 支持获取合约 ABI。
5. 支持解码交易 input 和事件 logs。
6. 将解码后的结构化数据发送给 LLM。
7. 输出一段 Markdown 格式交易解释。
8. 严格区分链上事实、模型推断和不确定信息。
9. 不实现钱包连接、不实现签名、不实现发交易。